RAPPORT FINAL DE QUALIFICATION — CreditRisk-Test

1. Synthèse exécutive

La campagne de qualification porte sur l'API de prédiction du risque de défaut de crédit : validation des données, fonctionnalités HTTP, intégration du pipeline, robustesse, non-régression, performance et intégration continue. La vérification finale sur la branche main, sous Windows et Python 3.13.2, a confirmé 56 tests réussis sur 56 en 2,26 secondes, ainsi qu'un contrôle Ruff sans erreur. Un avertissement StarletteDeprecationWarning demeure non bloquant.

La dernière couverture globale documentée par GitHub Actions est de 91 %. Les quatre campagnes Locust totalisent 10 418 requêtes, sans échec, avec un P95 de 20 à 21 ms. Un test volontairement défaillant a également démontré que GitHub Actions marque le workflow en échec lorsque Pytest échoue (56 tests réussis, 1 échec intentionnel).

Avis : favorable à une démonstration pédagogique ; défavorable à une mise en production ou à l'utilisation pour des décisions financières réelles. Le rappel (recall) de 14,65 % sur données synthétiques est insuffisant pour établir l'aptitude métier.

2. Système évalué et périmètre

Le système repose sur FastAPI et scikit-learn, avec un modèle de régression logistique entraîné sur des données entièrement synthétiques. Les points d'accès évalués sont /health, /model-info et /predict. Les étapes couvertes sont : validation des entrées, prétraitement, chargement du modèle, inférence, réponse HTTP, traitement des erreurs et reproductibilité.

Hors périmètre : validation sur dossiers réels, calibration, équité entre groupes, surveillance de dérive, sécurité opérationnelle, essais d'endurance et qualification d'une infrastructure de production.

3. Environnement, méthode et preuves

Domaine	Environnement / méthode	Preuves
Tests locaux	Windows 11 Pro 64 bits ; Python 3.13.2 ; Pytest 9.2.0	56/56 réussis ; sortie terminal finale
Machine de test	Intel Core Ultra 7 165H ; 16 cœurs, 22 processeurs logiques ; 63,45 Go RAM	Commandes PowerShell Get-CimInstance
API et modèle	FastAPI 0.162.2 ; scikit-learn 1.9.1	tests/ ; scripts/train_model.py
Charge	Locust 2.47.0 ; campagnes HTTP locales	reports/load_*_stats.csv ; rapport de performance
CI	GitHub Actions ; Ubuntu ; Python 3.12 ; Ruff ; pytest-cov	Runs réussis sur main et run d'échec #42700994044
Traçabilité	Plan de test, matrice, registre des risques	reports/plan_de_test.md ; reports/matrice_tracabilite.md

Les versions de Python diffèrent entre le poste local (3.13.2) et la CI (3.12). Les mesures TestClient et Locust proviennent de protocoles distincts et ne doivent pas être assimilées.

4. Résultats des tests automatisés

Catégorie	Réussis	Échecs
Fonctionnels	17	0
Intégration	11	0
Robustesse	18	0
Non-régression (test_regression.py)	7	0
Performance automatisée	2	0
Métriques du modèle	1	0
TOTAL	56	0

La famille non-régression représente 8 contrôles si l'on inclut le test distinct des métriques du modèle. Elle ne doit pas être additionnée une seconde fois au total de 56. Le test de rechargement vérifie l'égalité des probabilités après sérialisation joblib, avec une tolérance absolue de 10^{-10}. Les scénarios de robustesse sont tous réussis (18/18).

La couverture globale CI documentée est de 91 %, sans garantie d'exhaustivité des comportements ni des risques métier. Un avertissement de dépréciation Starlette/TestClient reste à surveiller.

5. Résultats de performance

5.1 Mesures locales TestClient

Indicateur	Mesure
Latence API moyenne	4,75 ms
Latence API P95	5,53 ms
Débit séquentiel estimé	210,60 req/s
Latence d'inférence P95	0,36 ms

5.2 Campagnes de charge Locust

Utilisateurs	Requêtes	Échecs	Débit (req/s)	P95	P99
5	191	0	3,24	20 ms	810 ms
20	767	0	12,90	20 ms	24 ms
50	1 895	0	31,88	21 ms	31 ms
100	7 565	0	63,62	20 ms	30 ms
TOTAL	10 418	0	—	—	—

Les scénarios de 5, 20 et 50 utilisateurs étaient programmés pour 60 secondes ; celui de 100 utilisateurs pour 120 secondes. Les utilisateurs avaient un temps d'attente aléatoire de 1 à 2 secondes. Les débits ne s'additionnent pas. Un pic atypique de 1 818,21 ms à cinq utilisateurs demeure inexpliqué.

Critères pédagogiques proposés : taux d'échec de 0 %, P95 inférieur à 100 ms et débit d'au moins 50 req/s à 100 utilisateurs. Ils sont satisfaits dans ces essais locaux, sans constituer des objectifs contractuels de production. Le rapport ne documente pas une surveillance CPU/RAM pendant les campagnes ni une campagne d'endurance.

6. Traçabilité et non-régression

La matrice de traçabilité comprend 20 exigences et doit conserver la correspondance avec les sept exigences obligatoires EX01–EX07 de l'énoncé. Les tests existants couvrent la validation, l'intégration, les erreurs, la robustesse, la performance et la CI.

Le contrôle réel test_model_metrics_against_baseline a été exécuté avec succès : rappel courant = 0,1465, rappel de référence = 0,1465 ; F1 courant = 0,2473, F1 de référence = 0,2473. La tolérance absolue est de 0,02 pour chaque métrique ; aucune dégradation au-delà de cette tolérance n'a été constatée sur le jeu synthétique reproductible de 1 000 observations (seed 42).

Deux tests supplémentaires vérifient la détection logique de baisses simulées de cinq points de pourcentage. Limite : aucune comparaison entre deux artefacts de modèles distincts (V1 et V2) n'a été démontrée. L'exigence historique EX-19 est donc partiellement couverte : comparaison du modèle chargé à la baseline validée, évolution réelle de version non démontrée.

7. Anomalies, risques et réserves

Observation	Constat	Suite recommandée
Robustesse	18 tests réussis ; aucune anomalie fonctionnelle détectée par ces assertions	Conserver le registre ; préciser la réponse observée pour le montant extrême
Dépendances	StarletteDeprecationWarning (httpx / TestClient)	Surveiller les compatibilités et mises à jour
Latence atypique	Maximum de 1 818,21 ms au scénario 5 utilisateurs	Répéter et instrumenter le test
Non-régression	Baseline vérifiée, pas de comparaison V1/V2 distinctes	Comparer des artefacts versionnés si requis
Qualité prédictive	Recall 14,65 %, F1 24,73 %, données synthétiques	Évaluer faux négatifs, calibration et groupes sur données représentatives
Couverture et charge	91 % de couverture ; pas de suivi ressources ni endurance	Compléter les scénarios non couverts et l'observabilité

Un rappel de 14,65 % signifie qu'environ 85,35 % des défauts présents dans ce jeu de validation ne sont pas détectés, sous l'hypothèse que la classe positive désigne effectivement le défaut. Ce risque interdit de conclure à une aptitude métier à partir des seuls tests logiciels.

8. Intégration continue et décision de qualification

Le workflow GitHub Actions est déclenché par push, pull_request ou manuellement. Il installe Python 3.12 et les dépendances, entraîne le modèle synthétique, exécute Ruff, lance Pytest avec couverture et archive les rapports XML. Les exécutions récentes de main sont réussies.

Démonstration de blocage CI : sur la branche temporaire demo/ci-test-failure, une première tentative a échoué au contrôle Ruff ; après correction de la forme de l'assertion, Ruff a réussi et Pytest a produit 56 passed, 1 failed, 1 warning, couverture 91 %, puis un code de sortie 1. La preuve distante est l'exécution GitHub Actions 42700994044 (https://github.com/kagibru/CreditRisk-Test/actions/runs/42700994044). Ce test volontaire n'a pas été fusionné dans main.

Critère	Résultat	Appréciation
Tests sur main	56/56 ; Ruff conforme	Satisfait
Couverture CI	91 %	Satisfait au seuil pédagogique 80 %
Échec CI sur test défaillant	Run 42700994044 ; exit code 1	Démontré
Charge locale	10 418 requêtes ; 0 échec ; P95 20–21 ms	Satisfait dans les conditions testées
Non-régression	Baseline réelle vérifiée ; V1/V2 non comparées	Partiellement démontré
Production bancaire	Données synthétiques ; recall faible	Non qualifié

DÉCISION : qualification favorable pour une démonstration pédagogique du système de tests et de l'API ; qualification défavorable pour toute utilisation opérationnelle dans une institution financière. La réussite technique ne vaut pas validation de la pertinence des décisions de crédit.

9. Recommandations et clôture

- Conserver les preuves reproductibles : journaux GitHub Actions, rapports XML, couverture, CSV Locust, matrice de traçabilité et registre des anomalies.
- Comparer deux artefacts de modèles réellement distincts sur le même jeu de validation figé, avec seuils et critères d'acceptation documentés, si une évolution du modèle est introduite.
- Répéter les campagnes de charge, investiguer le pic de latence et instrumenter CPU, RAM et temps de réponse sur une infrastructure représentative.
- Avant tout usage métier : valider sur données réelles représentatives, analyser le taux de faux négatifs par profil, la calibration, les biais, la dérive et prévoir une supervision humaine.
- Mettre à jour les dépendances de manière contrôlée et conserver la preuve que le pipeline bloque un test défaillant.

Références internes

- reports/plan_de_test.md
- reports/matrice_tracabilite.md
- reports/registre_anomalies_robustesse.md
- reports/rapport_non_regression_creditrisk.md
- reports/rapport_performance_creditrisk.md
- reports/load_5_stats.csv, load_20_stats.csv, load_50_stats.csv, load_100_stats.csv
- reports/coverage.xml et rapports XML Pytest
- GitHub Actions : run 42700994044